

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №3.

Тема: «Решение задачи о коммивояжере»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

Требуется разработать программу для поиска оптимального маршрута коммивояжёра во взвешенном графе. Программа должна предусматривать:

- хранение весов рёбер в виде матрицы смежности;
- перебор всех возможных гамильтоновых циклов для нахождения кратчайшего пути;
- наглядный вывод информации в консоль;
- отображение итогового маршрута в виде вертикальной схемы (дерева).

Граф должен включать не менее шести вершин. Это необходимо, чтобы:

- показать, что количество возможных маршрутов достаточно велико;
- продемонстрировать сравнение разных вариантов обхода;
- наглядно выделить маршрут с минимальной стоимостью.

Анализ задачи:

Рассматривается граф, заданный исходной матрицей весов. Процесс решения включает следующие этапы:

1. Запись исходных данных в матрицу смежности.
2. Приведение матрицы по строкам (редукция строк).
3. Приведение матрицы по столбцам (редукция столбцов).
4. Вычисление штрафов для всех нулевых элементов.
5. Определение наиболее перспективного ребра для включения в маршрут.
6. Исключение из дальнейшего рассмотрения строки и столбца, соответствующих выбранному ребру.
7. Повторение шагов 2–6 до тех пор, пока не будет построен полный маршрут.
8. Формирование окончательного пути и его вывод.

Код:

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

const int VERTICES = 6;
const int MAX_DIST = 999999;

char labels[VERTICES] = { 'A', 'B', 'C', 'D', 'E', 'F' };
```

```

int distances[VERTICES][VERTICES] =
{
    {MAX_DIST, 12, 7, 18, 5, 9},
    {12, MAX_DIST, 4, 6, 14, 11},
    {7, 4, MAX_DIST, 8, 10, 15},
    {18, 6, 8, MAX_DIST, 3, 13},
    {5, 14, 10, 3, MAX_DIST, 16},
    {9, 11, 15, 13, 16, MAX_DIST}
};

struct RouteSegment
{
    int start;
    int end;
};

vector<RouteSegment> solution;

void showMatrix(int mat[VERTICES][VERTICES])
{
    cout << "\n--- ТЕКУЩАЯ МАТРИЦА ---\n\n";

    cout << setw(8) << " ";

    for (int i = 0; i < VERTICES; i++)
        cout << setw(6) << labels[i];

    cout << endl;

    for (int i = 0; i < VERTICES; i++)
    {
        cout << setw(6) << labels[i];

        for (int j = 0; j < VERTICES; j++)
        {
            if (mat[i][j] == MAX_DIST)
                cout << setw(6) << "INF";
            else
                cout << setw(6) << mat[i][j];
        }

        cout << endl;
    }
}

int rowOptimization(int mat[VERTICES][VERTICES])
{
    int totalReduction = 0;

    for (int i = 0; i < VERTICES; i++)
    {
        int minRow = MAX_DIST;

        for (int j = 0; j < VERTICES; j++)
        {
            if (mat[i][j] < minRow)
                minRow = mat[i][j];
        }

        if (minRow != MAX_DIST && minRow != 0)
        {
            totalReduction += minRow;

            for (int j = 0; j < VERTICES; j++)
            {
                if (mat[i][j] != MAX_DIST)
                    mat[i][j] -= minRow;
            }
        }
    }
}

```

```

    }
}

return totalReduction;
}

int columnOptimization(int mat[VERTICES][VERTICES])
{
    int totalReduction = 0;

    for (int j = 0; j < VERTICES; j++)
    {
        int minCol = MAX_DIST;

        for (int i = 0; i < VERTICES; i++)
        {
            if (mat[i][j] < minCol)
                minCol = mat[i][j];
        }

        if (minCol != MAX_DIST && minCol != 0)
        {
            totalReduction += minCol;

            for (int i = 0; i < VERTICES; i++)
            {
                if (mat[i][j] != MAX_DIST)
                    mat[i][j] -= minCol;
            }
        }
    }

    return totalReduction;
}

int estimateZeroPenalty(int mat[VERTICES][VERTICES], int r, int c)
{
    int smallestRow = MAX_DIST;
    int smallestCol = MAX_DIST;

    for (int j = 0; j < VERTICES; j++)
    {
        if (j != c && mat[r][j] < smallestRow)
            smallestRow = mat[r][j];
    }

    for (int i = 0; i < VERTICES; i++)
    {
        if (i != r && mat[i][c] < smallestCol)
            smallestCol = mat[i][c];
    }

    if (smallestRow == MAX_DIST) smallestRow = 0;
    if (smallestCol == MAX_DIST) smallestCol = 0;

    return smallestRow + smallestCol;
}

void displayZeroPenalties(int mat[VERTICES][VERTICES])
{
    cout << "\n--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---\n";

    for (int i = 0; i < VERTICES; i++)
    {
        for (int j = 0; j < VERTICES; j++)
        {
            if (mat[i][j] == 0)
            {
                int penalty = estimateZeroPenalty(mat, i, j);
            }
        }
    }
}

```

```

        cout << "[" << labels[i] << " -> " << labels[j] << "] штраф = " <<
penalty << endl;
    }
}
cout << endl;
}

RouteSegment selectBestTransition(int mat[VERTICES][VERTICES])
{
    int maxPenalty = -1;
    RouteSegment best = { -1, -1 };

    for (int i = 0; i < VERTICES; i++)
    {
        for (int j = 0; j < VERTICES; j++)
        {
            if (mat[i][j] == 0)
            {
                int penalty = estimateZeroPenalty(mat, i, j);

                if (penalty > maxPenalty)
                {
                    maxPenalty = penalty;
                    best.start = i;
                    best.end = j;
                }
            }
        }
    }

    if (best.start == -1 || best.end == -1)
    {
        cout << "Нулевые элементы не найдены.\n";
        return { -1, -1 };
    }

    cout << "\nВЫБРАНО РЕБРО: " << labels[best.start] << " -> " << labels[best.end] <<
endl;
    cout << "ВЕЛИЧИНА ШТРАФА: " << maxPenalty << endl;

    return best;
}

void eliminateRowColumn(int mat[VERTICES][VERTICES], int r, int c)
{
    for (int j = 0; j < VERTICES; j++)
        mat[r][j] = MAX_DIST;

    for (int i = 0; i < VERTICES; i++)
        mat[i][c] = MAX_DIST;

    mat[c][r] = MAX_DIST;
}

void showSolution()
{
    cout << "\n===== ПОЛУЧЕННЫЙ МАРШРУТ =====\n";

    for (int i = 0; i < solution.size(); i++)
    {
        cout << labels[solution[i].start] << " -> " << labels[solution[i].end];

        if (i < solution.size() - 1)
            cout << endl;
    }
    cout << "\n";
}

```

```

int computeTotalPathLength()
{
    int total = 0;

    for (auto seg : solution)
    {
        total += distances[seg.start][seg.end];
    }

    return total;
}

int main()
{
    setlocale(LC_ALL, "Russian");

    int workMatrix[VERTICES][VERTICES];

    for (int i = 0; i < VERTICES; i++)
        for (int j = 0; j < VERTICES; j++)
            workMatrix[i][j] = distances[i][j];

    cout << "\n";
    cout << "===== ЗАДАЧА КОММИВОЯЖЕРА =====\n";
    cout << "===== МЕТОД ВЕТВЕЙ И ГРАНИЦ =====\n";

    cout << "\nИСХОДНАЯ МАТРИЦА:\n";
    showMatrix(workMatrix);

    int globalLowerBound = 0;

    for (int iteration = 1; iteration <= VERTICES; iteration++)
    {
        cout << "\n===== ШАГ " << iteration << " =====\n";

        int rowReduce = rowOptimization(workMatrix);
        cout << "Редукция строк: " << rowReduce << endl;

        int colReduce = columnOptimization(workMatrix);
        cout << "Редукция столбцов: " << colReduce << endl;

        globalLowerBound += rowReduce + colReduce;

        cout << "Текущая нижняя граница: " << globalLowerBound << endl;

        showMatrix(workMatrix);

        displayZeroPenalties(workMatrix);

        RouteSegment chosen = selectBestTransition(workMatrix);

        if (chosen.start == -1)
            break;

        solution.push_back(chosen);

        eliminateRowColumn(workMatrix, chosen.start, chosen.end);

        cout << "\nДОБАВЛЕНО В МАРШРУТ: " << labels[chosen.start] << " -> " <<
labels[chosen.end] << "\n";
    }

    showSolution();

    cout << "ИТОГОВЫЙ МАРШРУТ:\n\n";

    for (auto seg : solution)
    {
        cout << labels[seg.start] << " -> ";
    }
}

```

```
    }  
  
    if (!solution.empty())  
        cout << labels[solution[0].start];  
  
    int totalCost = computeTotalPathLength();  
  
    cout << "\n\nОБЩАЯ СТОИМОСТЬ = " << totalCost << endl;  
  
    return 0;  
}
```

Вывод программы

===== ЗАДАЧА КОММИВОЯЖЕРА =====
===== МЕТОД ВЕТВЕЙ И ГРАНИЦ =====

ИСХОДНАЯ МАТРИЦА:

--- ТЕКУЩАЯ МАТРИЦА ---

	A	B	C	D	E	F
A	INF	12	7	18	5	9
B	12	INF	4	6	14	11
C	7	4	INF	8	10	15
D	18	6	8	INF	3	13
E	5	14	10	3	INF	16
F	9	11	15	13	16	INF

===== ШАГ 1 =====

Редукция строк: 28

Редукция столбцов: 4

Текущая нижняя граница: 32

--- ТЕКУЩАЯ МАТРИЦА ---

	A	B	C	D	E	F
A	INF	7	2	13	0	0
B	8	INF	0	2	10	3
C	3	0	INF	4	6	7
D	15	3	5	INF	0	6
E	2	11	7	0	INF	9
F	0	2	6	4	7	INF

--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---

[A -> E] штраф = 0

[A -> F] штраф = 3

[B -> C] штраф = 4

[C -> B] штраф = 5

[D -> E] штраф = 3

[E -> D] штраф = 4

[F -> A] штраф = 4

ВЫБРАНО РЕБРО: C -> B

ВЕЛИЧИНА ШТРАФА: 5

ДОБАВЛЕНО В МАРШРУТ: C -> B

===== ШАГ 2 =====

Редукция строк: 2

Редукция столбцов: 2

Текущая нижняя граница: 36

--- ТЕКУЩАЯ МАТРИЦА ---

Текущая нижняя граница: 36

--- ТЕКУЩАЯ МАТРИЦА ---

	A	B	C	D	E	F
A	INF	INF	0	13	0	0
B	6	INF	INF	0	8	1
C	INF	INF	INF	INF	INF	INF
D	15	INF	3	INF	0	6
E	2	INF	5	0	INF	9
F	0	INF	4	4	7	INF

--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---

[A -> C] штраф = 3

[A -> E] штраф = 0

[A -> F] штраф = 1

[B -> D] штраф = 1

[D -> E] штраф = 3

[E -> D] штраф = 2

[F -> A] штраф = 6

ВЫБРАНО РЕБРО: F -> A

ВЕЛИЧИНА ШТРАФА: 6

ДОБАВЛЕНО В МАРШРУТ: F -> A

===== ШАГ 3 =====

Редукция строк: 0

Редукция столбцов: 1

Текущая нижняя граница: 37

--- ТЕКУЩАЯ МАТРИЦА ---

	A	B	C	D	E	F
A	INF	INF	0	13	0	INF
B	INF	INF	INF	0	8	0
C	INF	INF	INF	INF	INF	INF
D	INF	INF	3	INF	0	5
E	INF	INF	5	0	INF	8
F	INF	INF	INF	INF	INF	INF

--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---

[A -> C] штраф = 3

[A -> E] штраф = 0

[B -> D] штраф = 0

[B -> F] штраф = 5

[D -> E] штраф = 3

[E -> D] штраф = 5

ВЫБРАНО РЕБРО: B -> F

ВЕЛИЧИНА ШТРАФА: 5

ВЫБРАНО РЕБРО: B -> F
ВЕЛИЧИНА ШТРАФА: 5

ДОБАВЛЕНО В МАРШРУТ: B -> F

===== ШАГ 4 =====

Редукция строк: 0

Редукция столбцов: 0

Текущая нижняя граница: 37

--- ТЕКУЩАЯ МАТРИЦА ---

	A	B	C	D	E	F
A	INF	INF	0	13	0	INF
B	INF	INF	INF	INF	INF	INF
C	INF	INF	INF	INF	INF	INF
D	INF	INF	3	INF	0	INF
E	INF	INF	5	0	INF	INF
F	INF	INF	INF	INF	INF	INF

--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---

[A -> C] штраф = 3

[A -> E] штраф = 0

[D -> E] штраф = 3

[E -> D] штраф = 18

ВЫБРАНО РЕБРО: E -> D

ВЕЛИЧИНА ШТРАФА: 18

ДОБАВЛЕНО В МАРШРУТ: E -> D

===== ШАГ 5 =====

Редукция строк: 3

Редукция столбцов: 0

Текущая нижняя граница: 40

--- ТЕКУЩАЯ МАТРИЦА ---

	A	B	C	D	E	F
A	INF	INF	0	INF	0	INF
B	INF	INF	INF	INF	INF	INF
C	INF	INF	INF	INF	INF	INF
D	INF	INF	0	INF	INF	INF
E	INF	INF	INF	INF	INF	INF
F	INF	INF	INF	INF	INF	INF

--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---

[A -> C] штраф = 0

[A -> E] штраф = 0

[D -> C] штраф = 0

ВЫБРАНО РЕБРО: A -> C

```

ВЫБРАНО РЕБРО: A -> C
ВЕЛИЧИНА ШТРАФА: 0

ДОБАВЛЕНО В МАРШРУТ: A -> C

===== ШАГ 6 =====
Редукция строк: 0
Редукция столбцов: 0
Текущая нижняя граница: 40

--- ТЕКУЩАЯ МАТРИЦА ---

  A   B   C   D   E   F
A INF INF INF INF INF INF
B INF INF INF INF INF INF
C INF INF INF INF INF INF
D INF INF INF INF INF INF
E INF INF INF INF INF INF
F INF INF INF INF INF INF

--- ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК ---
Нулевые элементы не найдены.

===== ПОЛУЧЕННЫЙ МАРШРУТ =====
C -> B
F -> A
B -> F
E -> D
A -> C
ИТОГОВЫЙ МАРШРУТ:
C -> F -> B -> E -> A -> C

ОБЩАЯ СТОИМОСТЬ = 34

C:\Users\Валерия\source\repos\ConsoleApplication52\x64\Debug\ConsoleApplication52.exe (процесс 16640) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

```

UML диаграмма

UML диаграмма классов Задача коммивояжера (Метод ветвей и границ)

